AI ASSISTED CODING

Assignment-15.1

Name:ORSU VENNELA

HT NO:2303A52418

BT No:35

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

• Endpoints required:

o GET /students → List all students

o POST /students → Add a new student

o PUT /students/{id} → Update student details

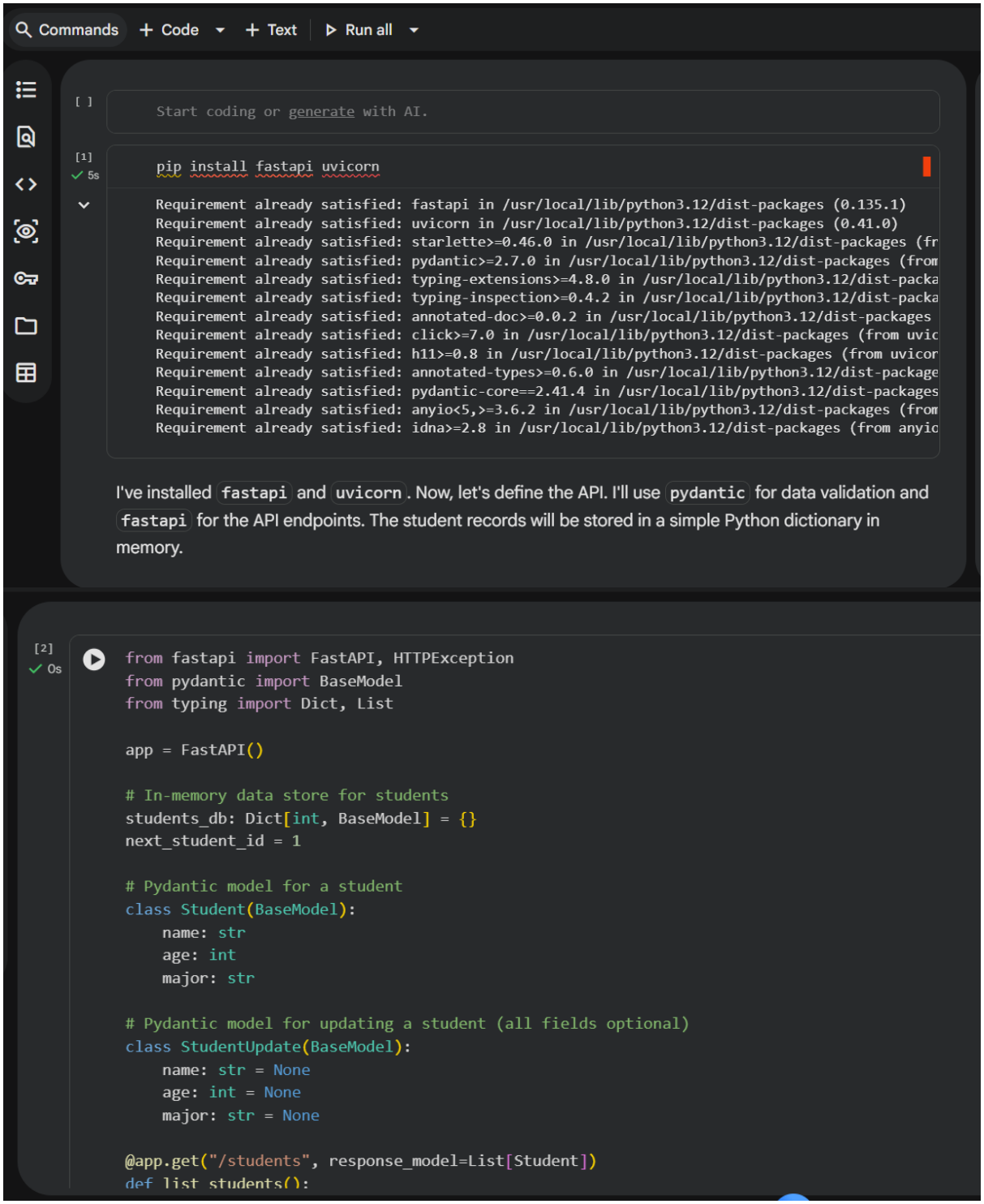
o DELETE /students/{id} → Delete a student record

• Use an in-memory data structure (list or dictionary) to store records.

• Ensure API responses are in JSON format.

Expected Output:

• Working API with CRUD functionality for student records.


Commands
+ Code
+ Text
Run all
[ ]
Start coding or generate with AI.
[1]
5s
pip install fastapi uvicorn
Requirement already satisfied: fastapi in /usr/local/lib/python3.12/dist-packages (0.135.1)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.41.0)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pydantic>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvic
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicor
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio
I've installed fastapi and uvicorn. Now, let's define the API. I'll use pydantic for data validation and fastapi for the API endpoints. The student records will be stored in a simple Python dictionary in memory.
[2]
0s
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Dict, List
app = FastAPI()
# In-memory data store for students
students_db: Dict[int, BaseModel] = {}
next_student_id = 1
# Pydantic model for a student
class Student(BaseModel):
name: str
age: int
major: str
# Pydantic model for updating a student (all fields optional)
class StudentUpdate(BaseModel):
name: str = None
age: int = None
major: str = None
@app.get("/students", response_model=List[Student])
def list_students():

[2] ✓ 0s

```python
def list_students():
    """List all students."""
    return list(students_db.values())

@app.post("/students", response_model=Student, status_code=201)
def add_student(student: Student):
    """Add a new student."""
    global next_student_id
    students_db[next_student_id] = student
    student_with_id = student.copy(update={"id": next_student_id})
    next_student_id += 1
    return student_with_id

@app.put("/students/{student_id}", response_model=Student)
def update_student(student_id: int, student_update: StudentUpdate):
    """Update student details."""
    if student_id not in students_db:
        raise HTTPException(status_code=404, detail="Student not found")

    existing_student = students_db[student_id]
    update_data = student_update.dict(exclude_unset=True)
    updated_student = existing_student.copy(update=update_data)
    students_db[student_id] = updated_student
    return updated_student
```

```python
    return updated_student

@app.delete("/students/{student_id}", status_code=204)
def delete_student(student_id: int):
    """Delete a student record."""
    if student_id not in students_db:
        raise HTTPException(status_code=404, detail="Student not found")
    del students_db[student_id]
    return {"message": "Student deleted successfully"}
```

Now that the API is defined, you can run it using Uvicorn. Since you are in a Colab environment, we need to use `nest_asyncio` to allow Uvicorn to run within the existing event loop. Also, the API will be accessible via `localhost:8000`. You can test it by running this cell and then using `curl` or a tool like Postman/Insomnia in a separate terminal or web browser.

To interact with the API, you can use the following in a new cell or a separate terminal:

- **GET all students:**

```
!curl http://localhost:8000/students
```

- **POST a new student:**

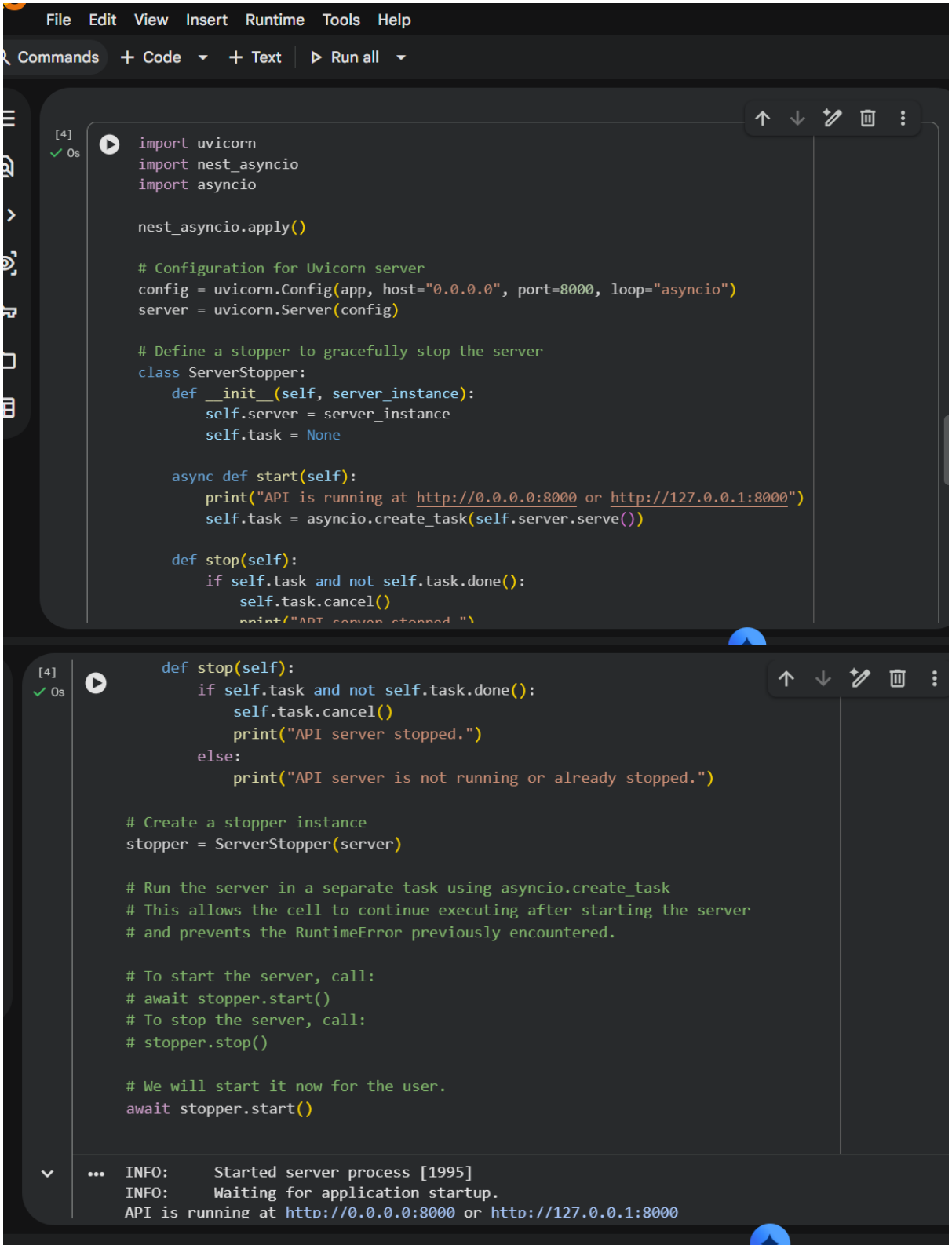
```
!curl -X POST -H "Content-Type: application/json" -d '{"name": "Alice", "age": 20, "major":
```

- **PUT (update) a student (assuming ID 1 exists):**

```
!curl -X PUT -H "Content-Type: application/json" -d '{"age": 21}' http://localhost:8000/stud
```

- **DELETE a student (assuming ID 1 exists):**

```
!curl -X DELETE http://localhost:8000/students/1
```



Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

• Endpoints required:

o GET /books → Retrieve all books

o POST /books → Add a new book

o GET /books/{id} → Get details of a specific book

o PATCH /books/{id} → Update partial book details

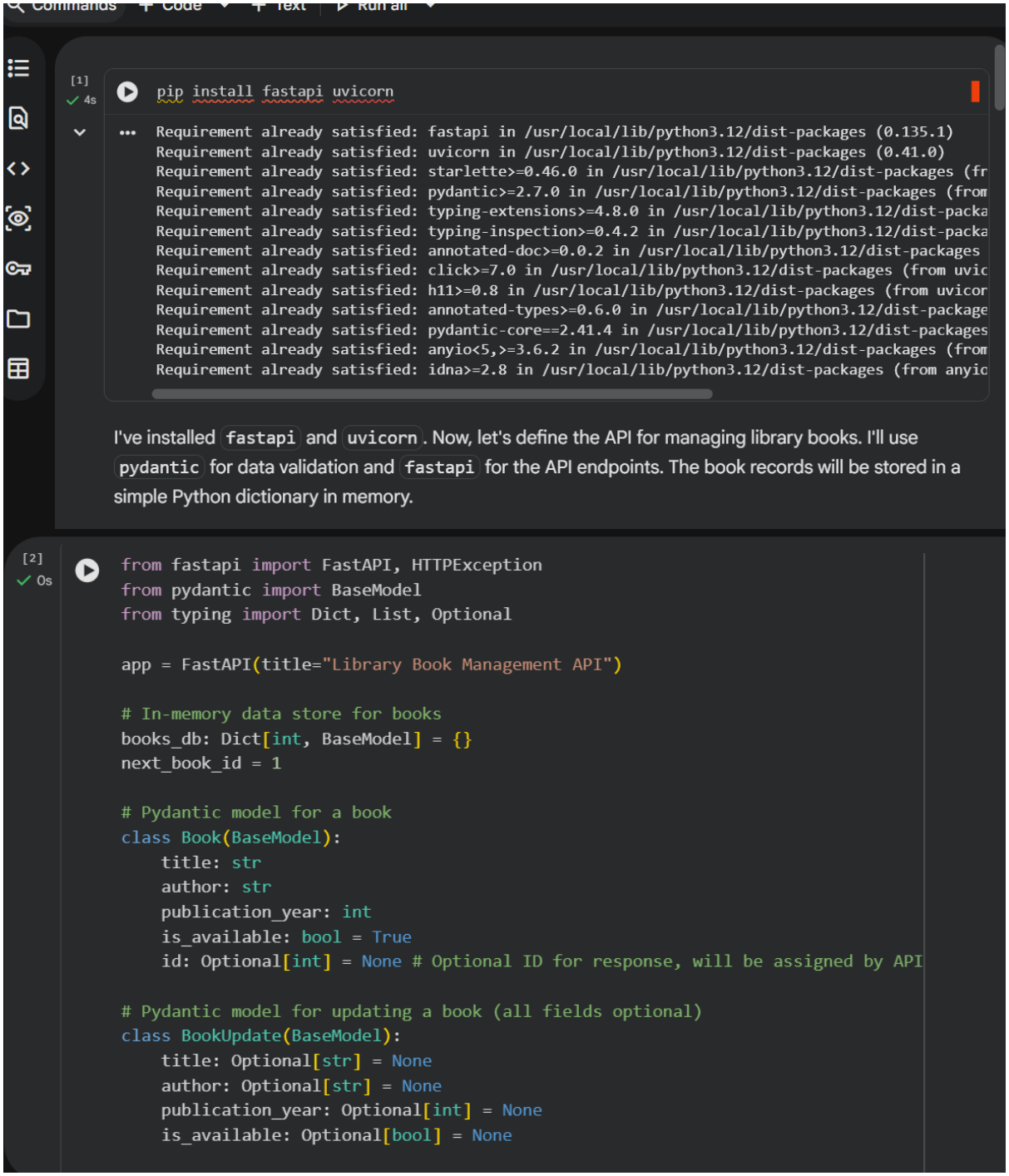
(e.g., availability)

o DELETE /books/{id} → Remove a book

• Implement error handling for invalid requests.

Expected Output:

• Functional API with CRUD + partial updates


Commands
+ Code
+ Text
Run all
[1]
4s
pip install fastapi uvicorn
Requirement already satisfied: fastapi in /usr/local/lib/python3.12/dist-packages (0.135.1)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.41.0)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pydantic>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvic
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicor
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio
I've installed fastapi and uvicorn. Now, let's define the API for managing library books. I'll use pydantic for data validation and fastapi for the API endpoints. The book records will be stored in a simple Python dictionary in memory.
[2]
0s
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Dict, List, Optional
app = FastAPI(title="Library Book Management API")
# In-memory data store for books
books_db: Dict[int, BaseModel] = {}
next_book_id = 1
# Pydantic model for a book
class Book(BaseModel):
title: str
author: str
publication_year: int
is_available: bool = True
id: Optional[int] = None # Optional ID for response, will be assigned by API
# Pydantic model for updating a book (all fields optional)
class BookUpdate(BaseModel):
title: Optional[str] = None
author: Optional[str] = None
publication_year: Optional[int] = None
is_available: Optional[bool] = None

[2] ✓ 0s

```python
@app.get("/books", response_model=List[Book], summary="Retrieve all books")
def get_all_books():
    """Retrieve a list of all books in the library."""
    return list(books_db.values())

@app.post("/books", response_model=Book, status_code=201, summary="Add a new book")
def add_book(book: Book):
    """Add a new book to the library.

    - **title**: The title of the book.
    - **author**: The author of the book.
    - **publication_year**: The year the book was published.
    - **is_available**: Whether the book is currently available (defaults to True).
    """
    global next_book_id
    book.id = next_book_id
    books_db[next_book_id] = book
    next_book_id += 1
    return book

@app.get("/books/{book_id}", response_model=Book, summary="Get details of a specific book")
def get_book_details(book_id: int):
    """Get details of a specific book by its ID."""
    if book_id not in books_db:
        raise HTTPException(status_code=404, detail="Book not found")
```

[2] ✓ 0s

```python
@app.patch("/books/{book_id}", response_model=Book, summary="Update partial book details")
def update_book_details(book_id: int, book_update: BookUpdate):
    """Update partial details of a book by its ID.

    - **title**: New title (optional).
    - **author**: New author (optional).
    - **publication_year**: New publication year (optional).
    - **is_available**: New availability status (optional).
    """
    if book_id not in books_db:
        raise HTTPException(status_code=404, detail="Book not found")

    existing_book = books_db[book_id]
    update_data = book_update.dict(exclude_unset=True) # Only update provided fields
    updated_book_data = existing_book.dict()
    updated_book_data.update(update_data)
    updated_book = Book(**updated_book_data)
    books_db[book_id] = updated_book
    return updated_book

@app.delete("/books/{book_id}", status_code=204, summary="Remove a book")
def delete_book(book_id: int):
    """Remove a book from the library by its ID."""
    if book_id not in books_db:
        raise HTTPException(status_code=404, detail="Book not found")
```

```
[2]  ✓ 0s
        del books_db[book_id]
        return {"message": "Book deleted successfully"}
```

Now that the API is defined, you can run it using Uvicorn. Since you are in a Colab environment, we need to use `nest_asyncio` to allow Uvicorn to run within the existing event loop. Also, the API will be accessible via `localhost:8000`. You can test it by running the following cell.

To stop the API server later, you can run `stopper.stop()` in a new cell.

To interact with the API, you can use `curl` commands in a new cell or a separate terminal:

- **GET all books:**

```
!curl http://localhost:8000/books
```

- **POST a new book:**

```
!curl -X POST -H "Content-Type: application/json" -d '{"title": "The Hobbit", "author": "J.R
```

- **GET a specific book (e.g., ID 1):**


```
!curl -X POST -H "Content-Type: application/json" -d '{"title": "The Hobbit", "author": "J.R
```

- **GET a specific book (e.g., ID 1):**


```
!curl http://localhost:8000/books/1
```

- **PATCH (update) book details (e.g., change availability for ID 1):**

```
!curl -X PATCH -H "Content-Type: application/json" -d '{"is_available": false}' http://locall
```

- **DELETE a book (e.g., ID 1):**

```
[3]
✓ 0s
import uvicorn
import nest_asyncio
import asyncio

nest_asyncio.apply()

# Configuration for Uvicorn server
config = uvicorn.Config(app, host="0.0.0.0", port=8000, loop="asyncio")
server = uvicorn.Server(config)

# Define a stopper to gracefully stop the server
class ServerStopper:
    def __init__(self, server_instance):
        self.server = server_instance
        self.task = None

    async def start(self):
        print("Library Book Management API is running at http://0.0.0.0:8000 or http://127.0.0
        self.task = asyncio.create_task(self.server.serve())

    def stop(self):
        if self.task and not self.task.done():
            self.task.cancel()
            print("API server stopped.")
        else:
            print("API server is not running or already stopped.")
```

```
[3]
✓ 0s
    def stop(self):
        if self.task and not self.task.done():
            self.task.cancel()
            print("API server stopped.")
        else:
            print("API server is not running or already stopped.")

# Create a stopper instance
stopper = ServerStopper(server)

# Start the server
await stopper.start()

INFO:     Started server process [3359]
INFO:     Waiting for application startup.
Library Book Management API is running at http://0.0.0.0:8000 or http://127.0.0.1:8000
+ Code  + Text
Add text cell
```

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

• Endpoints required:

o GET /employees → List all employees

o POST /employees → Add a new employee with

salary details

o PUT /employees/{id}/salary → Update salary of an

employee

o DELETE /employees/{id} → Remove employee

from system

• Use AI to:

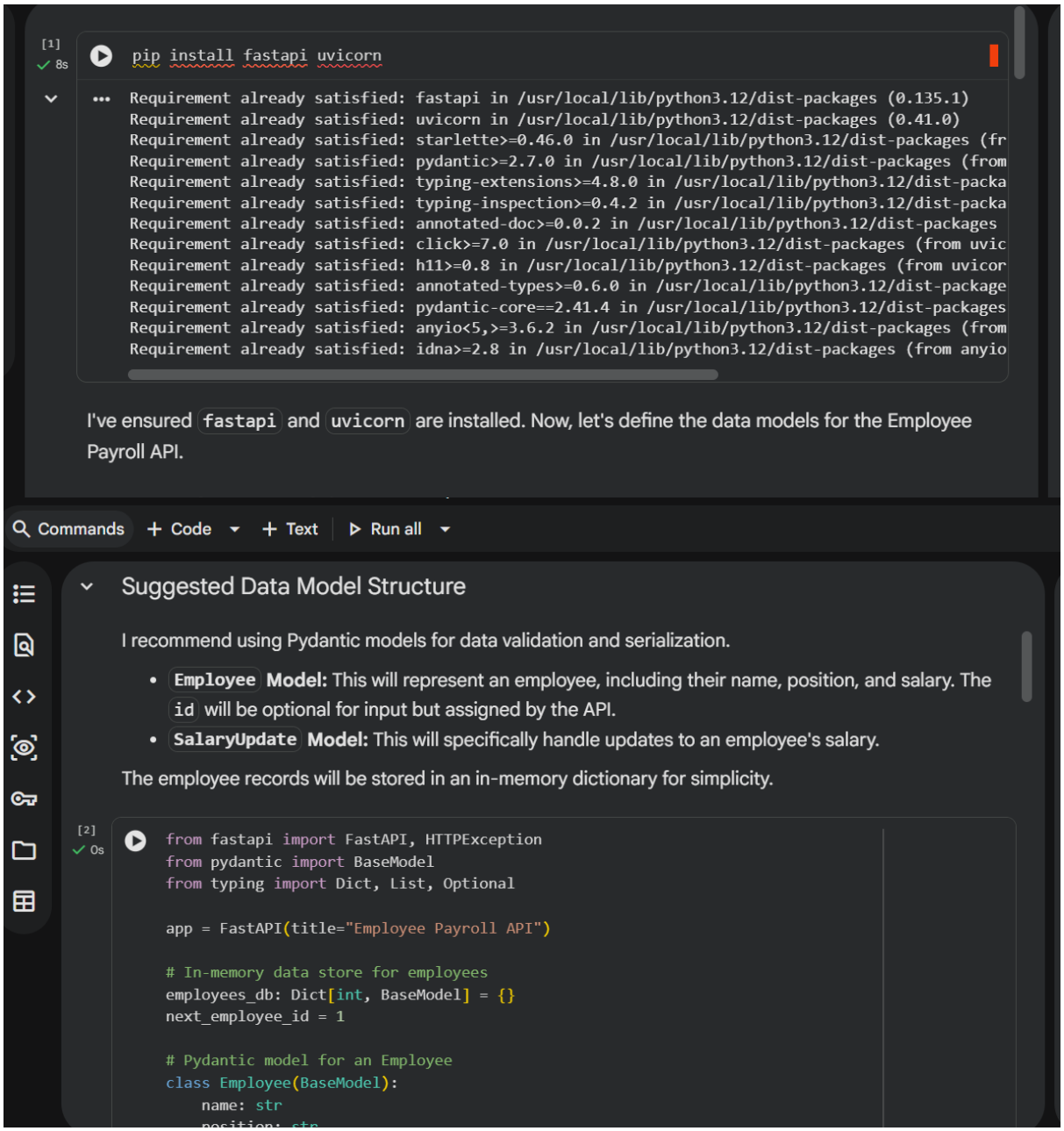
o Suggest data model structure.

o Add comments/docstrings for all endpoints.

Expected Output:

• API supporting salary management with clear

documentation.



```
0s
class Employee(BaseModel):
    name: str
    position: str
    salary: float
    id: Optional[int] = None  # Optional ID for response, will be assigned by API

# Pydantic model for updating only the salary
class SalaryUpdate(BaseModel):
    salary: float

@app.get("/employees", response_model=List[Employee], summary="List all employees")
def list_employees():
    """Retrieve a list of all employees in the system, including their details."""
    return list(employees_db.values())

@app.post("/employees", response_model=Employee, status_code=201, summary="Add a new employee"
def add_employee(employee: Employee):
    """Add a new employee to the system with their name, position, and salary.

    - **name**: The full name of the employee.
    - **position**: The job title or position of the employee.
    - **salary**: The current salary of the employee.
    """
```

```
[2]
✓ 0s
    global next_employee_id
    employee.id = next_employee_id
    employees_db[next_employee_id] = employee
    next_employee_id += 1
    return employee

@app.put("/employees/{employee_id}/salary", response_model=Employee, summary="Update salary of
def update_employee_salary(employee_id: int, salary_update: SalaryUpdate):
    """Update the salary of a specific employee identified by their ID.

    - **employee_id**: The unique identifier of the employee.
    - **salary**: The new salary value for the employee.
    """
    if employee_id not in employees_db:
        raise HTTPException(status_code=404, detail="Employee not found")

    existing_employee = employees_db[employee_id]
    existing_employee.salary = salary_update.salary
    employees_db[employee_id] = existing_employee
    return existing_employee

@app.delete("/employees/{employee_id}", status_code=204, summary="Remove an employee")
def delete_employee(employee_id: int):
    """Remove an employee record from the system by their ID.
```

```
[2]
✓ 0s
    """
    if employee_id not in employees_db:
        raise HTTPException(status_code=404, detail="Employee not found")
    del employees_db[employee_id]
    return {"message": "Employee deleted successfully"}
```

Now that the Employee Payroll API is defined, you can run it using Uvicorn. Since you are in a Colab environment, we need to use `nest_asyncio` to allow Uvicorn to run within the existing event loop. The API will be accessible via `localhost:8000`. You can test it by running the following cell.

To stop the API server later, you can run `stopper.stop()` in a new cell.

To interact with the API, you can use `curl` commands in a new cell or a separate terminal:

- **GET all employees:**

```
!curl http://localhost:8000/employees
```

- **POST a new employee:**

```
!curl -X POST -H "Content-Type: application/json" -d '{"name": "John Doe", "position": "Soft
```

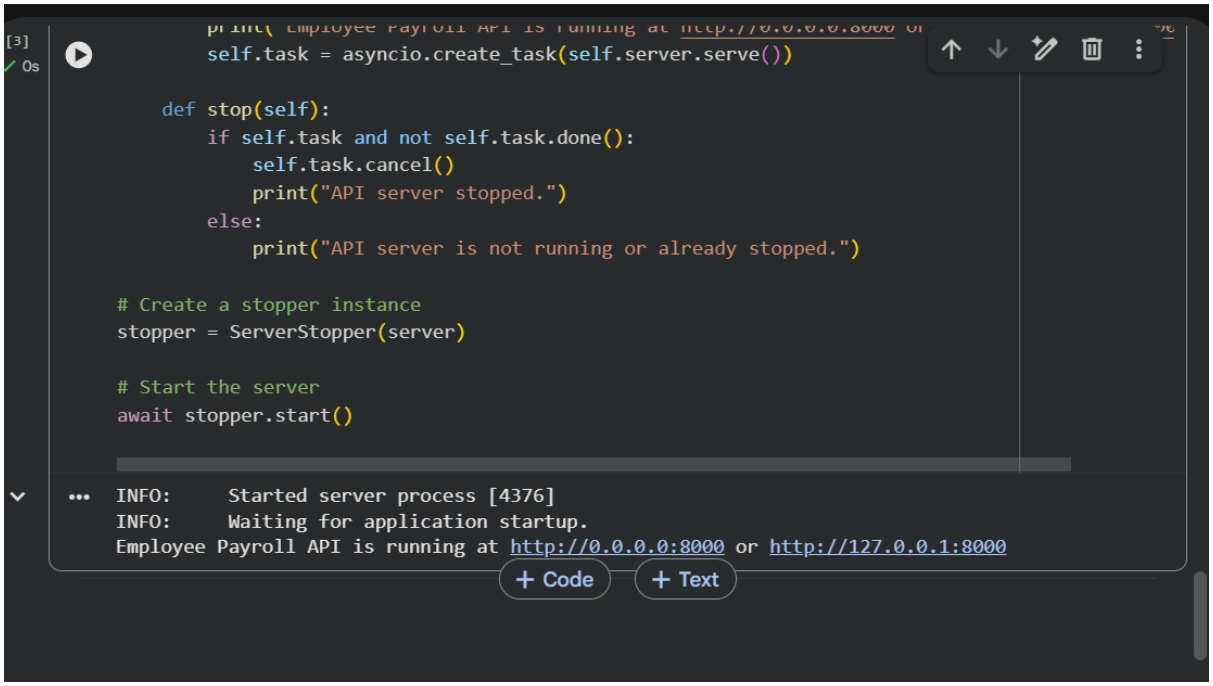
- **PUT (update) an employee's salary (e.g., for employee ID 1):**

```
!curl -X PUT -H "Content-Type: application/json" -d '{"salary": 80000.0}' http://localhost:8
```

- **DELETE an employee (e.g., employee ID 1):**

```
!curl -X DELETE http://localhost:8000/employees/1
```

```
[3]
✓ 0s
import uvicorn
import nest_asyncio
import asyncio

nest_asyncio.apply()

# Configuration for Uvicorn server
config = uvicorn.Config(app, host="0.0.0.0", port=8000, loop="asyncio")
server = uvicorn.Server(config)

# Define a stopper to gracefully stop the server
class ServerStopper:
    def __init__(self, server_instance):
        self.server = server_instance
        self.task = None

    async def start(self):
        print("Employee Payroll API is running at http://0.0.0.0:8000 or http://127.0.0.1:8000
        self.task = asyncio.create_task(self.server.serve())

    def stop(self):
        if self.task and not self.task.done():
            self.task.cancel()
            print("API server stopped.")
        else:
```



Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

• Endpoints required:

o GET /menu → List available dishes

o POST /order → Place a new order

o GET /order/{id} → Track order status

o PUT /order/{id} → Update an existing order (e.g.,

change items)

o DELETE /order/{id} → Cancel an order

• AI should generate:

o REST API code

o Suggested improvements (like authentication,

pagination)

Expected Output:

• Fully working API simulating a food ordering backend.

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.

2. At least 3 assert test cases for each task.

3. AI-generated initial code and execution screenshots.

4. Analysis of whether code passes all tests.

5. Improved final version with inline comments and explanation.

6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

```
[1]
✓ 6s
pip install fastapi uvicorn

Requirement already satisfied: fastapi in /usr/local/lib/python3.12/dist-packages (0.135.1)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.41.0)
Requirement already satisfied: starlette>=0.46.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pydantic>=2.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvic
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicor
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio
```

I've ensured `fastapi` and `uvicorn` are installed. Now, let's define the API for the online food ordering system. I'll use `pydantic` for data validation and `fastapi` for the API endpoints. Menu items and orders will be stored in in-memory dictionaries.

```
[2]
✓ 0s
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Dict, List, Optional
import uuid

app = FastAPI(title="Online Food Ordering API")

# In-memory data store for menu items and orders
menu_db: Dict[str, BaseModel] = {}
orders_db: Dict[str, BaseModel] = {}

# Initialize some sample menu items
menu_db["dish1"] = {"name": "Pizza Margherita", "description": "Classic pizza with tomato and
menu_db["dish2"] = {"name": "Spaghetti Carbonara", "description": "Pasta with eggs, hard chees
menu_db["dish3"] = {"name": "Caesar Salad", "description": "Fresh romaine lettuce with Caesar

# Pydantic model for a Dish
class Dish(BaseModel):
    name: str
    description: str
    price: float
    dish_id: Optional[str] = None # Added for clarity, will be key in menu_db

# Pydantic model for an item within an Order
class OrderItem(BaseModel):
```

```python
# Pydantic model for an item within an Order
class OrderItem(BaseModel):
    dish_id: str
    quantity: int

# Pydantic model for an Order
class Order(BaseModel):
    items: List[OrderItem]
    status: str = "pending"
    total_price: float = 0.0
    order_id: Optional[str] = None # Will be generated by the API

# Pydantic model for updating an existing order
class OrderUpdate(BaseModel):
    items: Optional[List[OrderItem]] = None
    status: Optional[str] = None


@app.get("/menu", response_model=List[Dish], summary="List available dishes")
def get_menu():
    """Retrieve a list of all available dishes on the menu."""
    # Convert menu_db values to Dish models and add dish_id
    dishes = []
    for dish_id, dish_data in menu_db.items():
        dish_obj = Dish(**dish_data)
        dishes.append(dish_obj)
    return dishes

@app.post("/order", response_model=Order, status_code=201, summary="Place a new order")
def place_order(order: Order):
    """Place a new order with a list of dishes and quantities.

    - **items**: A list of `OrderItem` objects, each containing `dish_id` and `quantity`.
    """
    order_id = str(uuid.uuid4()) # Generate a unique order ID
    total_price = 0.0

    # Calculate total price and validate dishes
    for item in order.items:
        if item.dish_id not in menu_db:
            raise HTTPException(status_code=404, detail=f"Dish '{item.dish_id}' not found in m
        dish_price = menu_db[item.dish_id]["price"]
        total_price += dish_price * item.quantity

    order.order_id = order_id
    order.total_price = total_price
    orders_db[order_id] = order
    return order

@app.get("/order/{order_id}", response_model=Order, summary="Track order status")
```

```
2]
✓ 0s
def get_order_status(order_id: str):
    """Track the status and details of a specific order by its ID."""
    if order_id not in orders_db:
        raise HTTPException(status_code=404, detail="Order not found")
    return orders_db[order_id]

@app.put("/order/{order_id}", response_model=Order, summary="Update an existing order")
def update_order(order_id: str, order_update: OrderUpdate):
    """Update an existing order's items or status.
    Note: This allows for significant changes, in a real app this might be more restricted.

    - **items**: An optional list of `OrderItem` objects to replace current items.
    - **status**: An optional string to update the order status (e.g., 'preparing', 'delivered
    """
    if order_id not in orders_db:
        raise HTTPException(status_code=404, detail="Order not found")

    existing_order = orders_db[order_id]
    update_data = order_update.dict(exclude_unset=True)

    # Update items if provided
    if "items" in update_data:
        new_total_price = 0.0
        for item in update_data["items"]:
            if item["dish_id"] not in menu_db:
```

```
[2]
✓ 0s
                raise HTTPException(status_code=404, detail=f"Dish '{item['dish_id']}' not fou
            dish_price = menu_db[item["dish_id"]]["price"]
            new_total_price += dish_price * item["quantity"]
        existing_order.items = [OrderItem(**item) for item in update_data["items"]]
        existing_order.total_price = new_total_price

    # Update status if provided
    if "status" in update_data:
        existing_order.status = update_data["status"]

    orders_db[order_id] = existing_order
    return existing_order

@app.delete("/order/{order_id}", status_code=204, summary="Cancel an order")
def cancel_order(order_id: str):
    """Cancel an existing order by its ID."""
    if order_id not in orders_db:
        raise HTTPException(status_code=404, detail="Order not found")

    # In a real system, you might change status to 'cancelled' instead of deleting
    del orders_db[order_id]
    return {"message": "Order cancelled successfully"}
```

Now that the Online Food Ordering API is defined, you can run it using Uvicorn. The API will be accessible via `localhost:8000`. You can test it by running the following cell.

To stop the API server later, you can run `stopper.stop()` in a new cell.

To interact with the API, you can use `curl` commands in a new cell or a separate terminal. Note that `order_id`s are UUIDs and will be generated by the `POST /order` endpoint.

- **GET menu:**

```
!curl http://localhost:8000/menu
```

- **POST a new order:**

```
!curl -X POST -H "Content-Type: application/json" -d '[{"dish_id": "dish1", "quantity": 2},
```

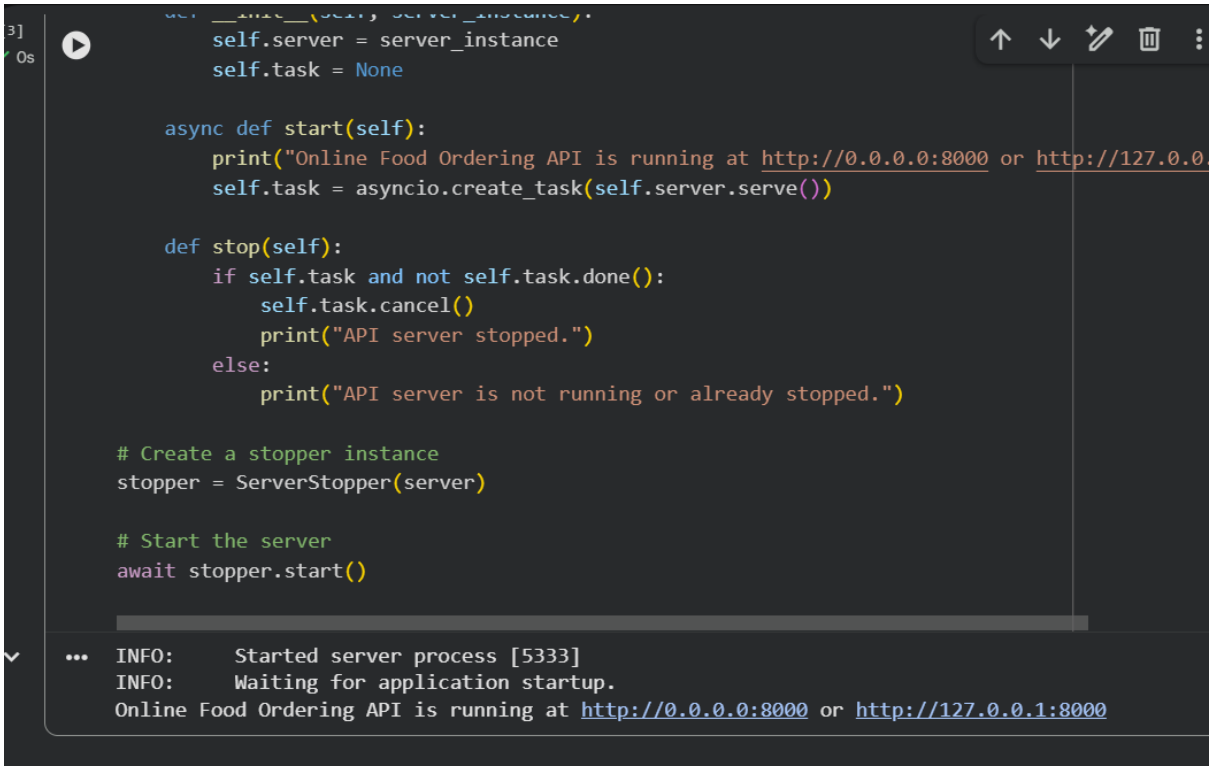
(Replace `dish_id` with actual IDs from `/menu`)

- **GET order status (replace `YOUR_ORDER_ID`):**

```
!curl http://localhost:8000/order/YOUR_ORDER_ID
```

[3]
✓ 0s

```python
import uvicorn
import nest_asyncio
import asyncio

nest_asyncio.apply()

# Configuration for Uvicorn server
config = uvicorn.Config(app, host="0.0.0.0", port=8000, loop="asyncio")
server = uvicorn.Server(config)

# Define a stopper to gracefully stop the server
class ServerStopper:
    def __init__(self, server_instance):
        self.server = server_instance
        self.task = None

    async def start(self):
        print("Online Food Ordering API is running at http://0.0.0.0:8000 or http://127.0.0.1:
        self.task = asyncio.create_task(self.server.serve())

    def stop(self):
        if self.task and not self.task.done():
```


self.server = server_instance
self.task = None
async def start(self):
print("Online Food Ordering API is running at http://0.0.0.0:8000 or http://127.0.0.
self.task = asyncio.create_task(self.server.serve())
def stop(self):
if self.task and not self.task.done():
self.task.cancel()
print("API server stopped.")
else:
print("API server is not running or already stopped.")
# Create a stopper instance
stopper = ServerStopper(server)
# Start the server
await stopper.start()
INFO: Started server process [5333]
INFO: Waiting for application startup.
Online Food Ordering API is running at http://0.0.0.0:8000 or http://127.0.0.1:8000